Assignment 1-Big environmental datasets & simple mass balances

Group 19
Anqi Yu, s254721
Jiayue Che, s252639
Yining Gan, s252690
Qiaowen You, s252709

12104 Environmental Modelling
23/09/2025

Contents

STEP I-System and data description ..... - 1 -
Task 1.1-System introduction ..... - 1 -
Task 1.2-Emissions of cumulative nitrogen pollutants ..... - 1 -
Task 1.3-Emission of micropollutants ..... - 2 -
STEP II-Model building ..... - 2 -
Task 2.1-Conceptualization description of the model ..... - 2 -
Task 2.2-Model diagnosis and testing ..... - 3 -
Task 2.3-Local sensitivity analysis ..... - 3 -
STEP III-Model application ..... - 4 -
Task 3.1-Simulation results ..... - 4 -
Task 3.2-Results Robustness ..... - 4 -
Task 3.3-Results Reliability ..... - 5 -
APPENDIX- STEP I-System and data description ..... - 6 -
APPENDIX-Task 1.1-System introduction ..... - 6 -
APPENDIX-Task 1.2-Emissions of cumulative nitrogen pollutants ..... - 10 -
APPENDIX-Task 1.3-Emission of micropollutants ..... - 12 -
APPENDIX-STEP II-Model building ..... - 13 -
APPENDIX-Task 2.1-Conceptualization description of the model ..... - 13 -
APPENDIX-Task 2.2-Model diagnosis and testing ..... - 16 -
APPENDIX-Task 2.3-Local sensitivity analysis ..... - 18 -
APPENDIX-STEP III-Model application ..... - 23 -
APPENDIX-Task 3.1-Simulation results ..... - 23 -
APPENDIX-Task 3.2-Results Robustness ..... - 24 -
APPENDIX-Task 3.3-Results Reliability ..... - 26 -
Group Declaration ..... - 27 -
Declare for Use of AI ..... - 27 -

STEP I-System and data description

Task 1.1-System introduction

The average flow of Mølle Å is 0.614 m³/s, with 0.187 m³/s of natural runoff added from Lyngby Lake to the sea. River flow is highest in January and lowest in August, so the greatest impact of CSO discharges is expected in January, when heavy winter rainfall and low evaporation overwhelmed wastewater treatment, resulting in maximum CSO releases.

Along Mølle Å, there are 23 CSO structures, shown as red dots in Figure 1. Over the past year, they discharged 636,156 m³ of water containing 7,589 kg of nitrogen. Ly R16 had the largest discharge, while Ly R4, Vi R2, Lu R20F, Lu R9, and Ly R16 represent the major CSO discharges, mostly located on the river's north bank. Downstream sections are likely to experience the most severe effects, as pollutants from the concentrated mid- and upper-reach CSO structures accumulate downstream.

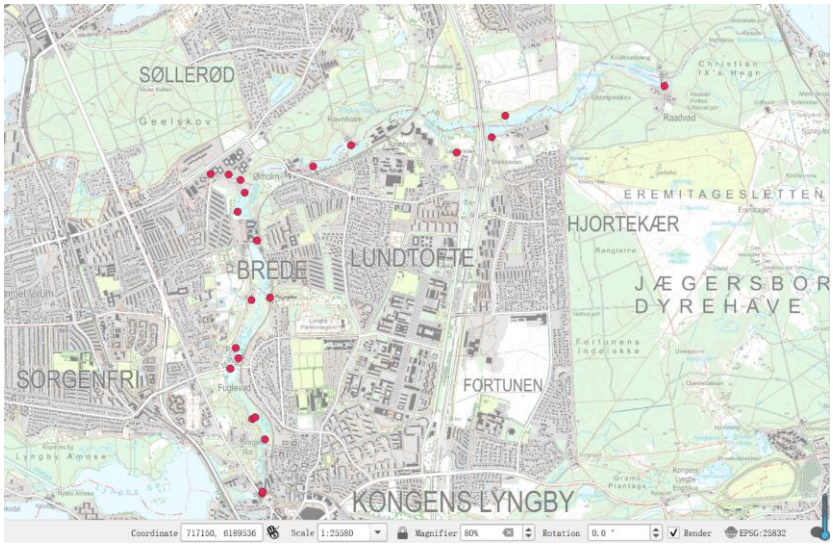


Figure 1 CSO structures are located along the Mølle Å stream

Task 1.2-Emissions of cumulative nitrogen pollutants

From 1990–2019, the average annual water volume[1] discharged from Lyngby Lake to Mølle Å was $1.74*10^7 m^3$, accounting for 73.7% of the outflow to the Øresund (Figure 2). Nitrogen loads are mainly from natural runoff (>45%), while CSO contributes over 40% (Figure 3, Figure 4) in both combined and separate sewer system. Converting the combined sewer system would reduce only ~293 kg N/year (~1% of outlet load) at high cost. Therefore, short-term, cost-effective measures—such as controlling improper discharges, installing small-scale CSO treatment, and targeted sewer upgrades—are recommended before considering full system conversion.

[1] The yearly values come from the 'vandfoering.aarsresultat' layer of the HIP model in Danmarks Miljøportalen (https://danmarksarealinformation.miljoeportal.dk).

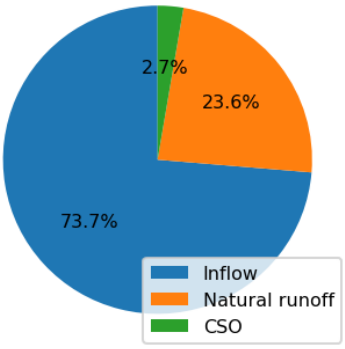


Figure 2 Composition of water volume sources discharged by Mølle Å river to the Øresund

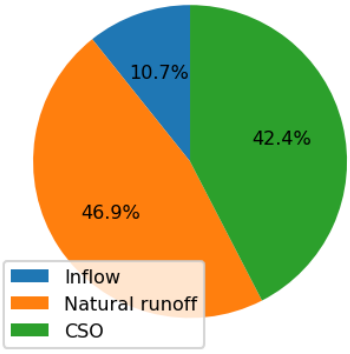


Figure 3 Composition of yearly Nitrogen load in the combined system

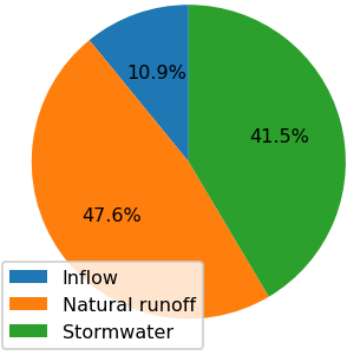


Figure 4 Composition of yearly Nitrogen load in the separate system

Task 1.3-Emission of micropollutants

The target micropollutant of our study is Benz(ghi)perylene. Benz(ghi)perylene concentrations in CSO show high variability (IQR/median = 2.75; 90% IQR/median = 4.75) [2], making mean or median values unrepresentative. Therefore, we apply the ML-estimated typical value as it reflects the pollutant level under normal conditions, providing more general and reliable insights into its impact on receiving water bodies. The 50% percentile reaches about twice the MAC-EQS, and the 95% exceeds it by over twelve times, suggesting a considerable water quality risk even under dilution.

In contrast, nitrogen concentrations in Lyngby Lake show much lower variability (CV=0.87), indicating temporal stability compared to the pronounced fluctuations of Benz(ghi)perylene in CSO.

STEP II-Model building

Task 2.1-Conceptualization description of the model

In this study, we simulate the MølleÅ River reach as a series of "completely mixed tanks," where water entering each node is instantly and fully mixed.

The Model in Action

Inputs: Real data from 'RiverData.csv' and 'csoData.csv'.

Parameters: Initial Benz(ghi)perylene concentration ($0.020 \mu g/L$ [3]); EQS; CSOs; Duration; 2.31% annual volume (other data from Table in file "Assignment 1").

Time & Space: Focuses on MølleÅ River (January 2019, most representative), chemical tracked at nodes with distances from IDs.

Scope: Covers MølleÅ River from upstream node "3687" to downstream node "13500".

Equation: $$C_i = \frac{Q_{i-1}C_{i-1} + Q_{CSO,i}C_{CSO,i}}{Q_i + Q_{CSO,i}}$$

where: Ci = pollutant concentration at node i (µg/L); Qi = river flow at node i (m³/h); QCSO,i = CSO flow at node i (m³/h); CCSO,i = pollutant concentration in CSO water (µg/L)

[2] Data source of percentiles: Miljøstyrelsen. (2022). *Typetal for miljøfarlige forurenende stoffer i regnbetingede udledninger på baggrund af data fra det nationale overvågningsprogram 2000 - 2020 NOVANA*. Bilag 3, p. 59. ISBN: 978 - 87 - 7038 - 386 - 8.

[3] Initial Benz(ghi)perylene concentration is referenced from "Typetal for miljøfarlige forurenende stoffer i regnbetingede Udledninger På baggrund af data fra det national overvågningsprogram 2000-2020".

Visualization

RiverC['conc']: Pollutant concentration at each node

EQS_exc['columnWithEQSexcValues']: Concentration values exceeding the EQS

Outputs can be used to visualize pollutant distribution along the river and CSO impacts.

Task 2.2-Model diagnosis and testing

The model results are consistent with my expectations. The key point, **Ly R16**, corresponding to the node **Novana_Model_MOELLEAA_DK1_4083.0(distance:396)**, is the CSO structure with the highest discharge volume. Therefore, it has the most significant impact on the microplastic concentration in the downstream area.

Downstream of this CSO, the concentration does show an increase. According to the simulation results, the concentration changes along the river—both in the upstream and downstream sections—are reasonable. This reflects the expected behavior of the system: the discharge from the CSO causes a local increase in concentration, while in the downstream area (if without interference from other external CSO facilities), the concentration gradually dilutes.

When doubling the pollutant concentration in the CSO led to a significant increase in downstream concentrations. This confirms that the model realistically reflects how local discharges influence downstream levels and how dilution reduces concentrations downstream, aligning with our expectations of pollutant transport.

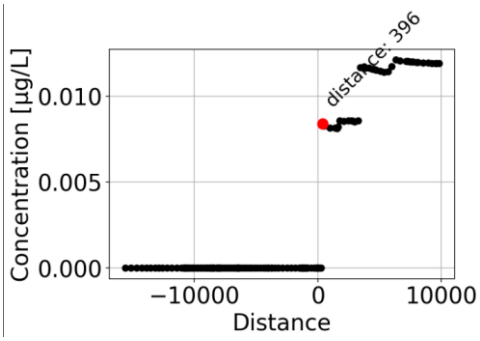


Figure 5 RiverC – Concentration vs. Distance

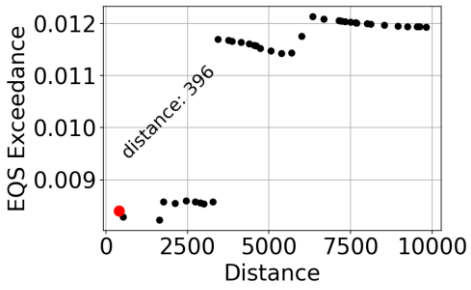


Figure 6 RiverC-EQS Exceedance vs. Distance

Task 2.3-Local sensitivity analysis

We can look at three two different model outputs: (1) The number of river nodes exceeding the EQS; (2) The max concentration; (3) pollutant loads discharged to the sea. Then we add a 10% to each parameter. The sensitivity indices results are showed below:

Table 1.Impact of Parameter Variations on Model Outputs

Changed\Si	Si_C_0	Si_EQS	Si_t_{CSO}	Si_θ	Si_ C_{CSO}
Output (1)	0.000	2.632	2.632	0.789	0.789
Output (2)	0.008	0.000	0.378	0.371	1.000
Output (3)	0.008	0.000	0.388	0.381	1.000

From the table data, it is obviously that environmental quality standards of EQS and Duration of a CSO “archetype event” are important for the number of river nodes exceeding the EQS; Micropollutant concentration in CSO water is the most important parameter for the max concentration and the pollutant loads discharged to the sea.

STEP III-Model application

Task 3.1-Simulation results

Based on the simulation using the selected micropollutant concentration of 0.018 µg/L. The simulated concentrations along the river range from near zero to approximately 0.011 µg/L, which exceeds the Environmental Quality Standard (EQS) of 8.2×10^{-3} µg/L in multiple sections.

From Figure 7 (Concentration vs. Distance): Upstream of the presumed CSO discharge point (distance < 0), contaminant concentration remains near 0, indicating no pollution. Downstream (distance > 0), concentration rises sharply, peaking and then stabilizing at a notably high level (approaching 0.0100 µg/L).

From Figure 8 (EQS Exceedance vs. Distance):"EQS Exceedance" (deviation from Environmental Quality Standards) is elevated across much of the downstream segment. After an initial decline (likely due to dilution or dispersion), exceedance rises again and remains high at longer distances (e.g., > 6000 m), confirming sustained chemical impairment.Thus, CSOs do pose a threat to the chemical status of Mølle Å 。

Based on the concentration profile in Figure 7, there are two distinct stretches of the Mølle Å that are at risk.

The locations of the two at-risk stretches are as follows: Primary Risk Stretch: This is the most severely impacted section, located between approximately 4,000 and 6,000 meters downstream. Here, the concentration not only exceeds the EQS but also reaches its peak. Secondary Risk Stretch: A second, shorter stretch is located further downstream, between approximately 7,250 and 8,000 meters (the end of the modeled section). The concentration rises again in this area, surpassing the EQS and indicating a renewed risk to water quality.

In conclusion, the simulation confirms that CSOs are a key threat to the Mølle Å, creating two non-compliant sections located between 4,000-6,000 m and 7,250-8,000 m.

Thisdiffers from the preliminary assessment (task 1.1), which predicted severe impacts only in the lower reaches。

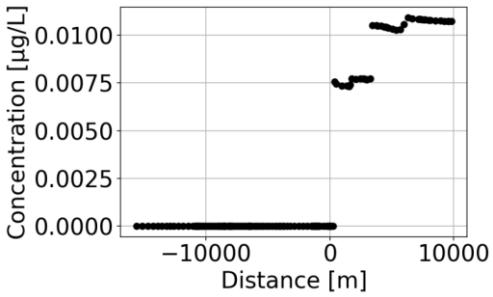


Figure 7 Mølle Å – Concentration vs. Distance

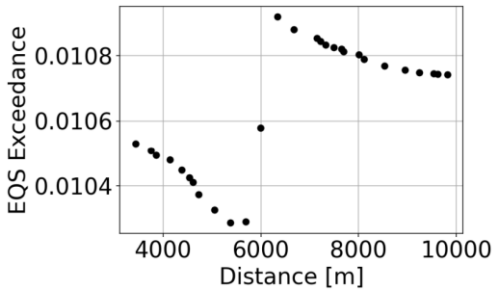


Figure 8 Mølle Å -EQS Exceedance vs. Distance

Task 3.2-Results Robustness

We run a Monte-Carlo simulation, where MP concentrations at CSOs structure are not fixed, but sampled from a lognormal distribution. For river nodes 0 to 3 and 120 to 138, there is a 95% probability of exceeding the Environmental Quality Standard (EQS); for nodes 110 to 138, there is a 50% probability of exceeding the EQS; for nodes 97 to 110, there is a 5% probability of exceeding the EQS; whereas for nodes 3 to 97, the standard is never exceeded.

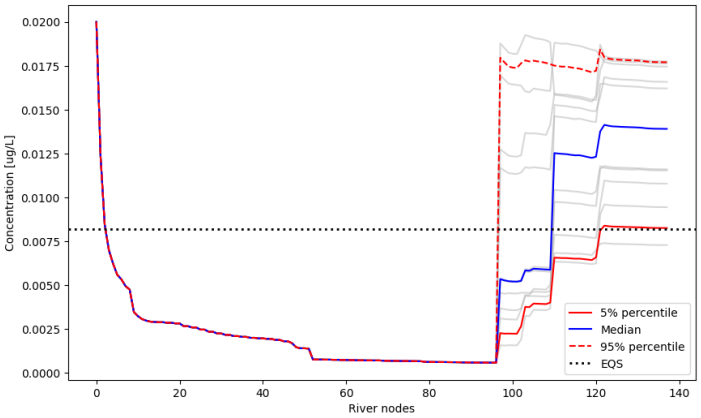


Figure 9 Monte Carlo simulations

Task 3.3-Results Reliability

The discharges of untreated wastewater from CSOs are indeed problematic for the Mølle Å stream. Based on our simulations, pollutant concentrations exceed the EQS in multiple stretches, indicating a potential risk to aquatic life. Since the EQS is designed to protect against acute effects, these exceedances could harm sensitive organisms, such as fish and invertebrates, through short-term exposure. The impacts may include reduced survival, growth impairment, or behavioral changes. Therefore, the CSO emissions pose a clear threat to the stream's ecological health.

Lyngby Municipality should consider investing in solutions to mitigate CSO impacts, especially in high-exceedance zones. However, the cost-effectiveness of such measures should be evaluated based on the severity and frequency of exceedances.

Validity of the Results

The validity of the results is moderately high but has limitations due to model assumptions and choices:

Model Choices: The model used a fixed concentration of 0.018 µg/L for CSO inputs in initial simulations, which was based on typical data but may not capture temporal variability. The Monte Carlo simulation improved robustness by sampling from a lognormal distribution, accounting for uncertainty in CSO concentrations. This shows that exceedances are probabilistic and not arbitrary.

Applicability to Other Pollutants: The results are specific to the selected micropollutant with an EQS of $8.2\times10-38.2\times10-3$ µg/L. For other pollutants with different EQS values, degradation rates, or toxicity, the results may not directly apply. For instance, pollutants with higher EQS might not exceed standards, while those with lower EQS could show worse impacts. Thus, the model should be recalibrated for each pollutant.

General Validity: The model captures spatial patterns well but simplifies hydrology, weather conditions, and pollutant transport. It assumes steady-state conditions, which may not reflect dynamic events like heavy rainfall. Nonetheless, the exceedance patterns align with CSO locations, adding credibility.

Suggestions for Improvement

To improve reliability, future work should:

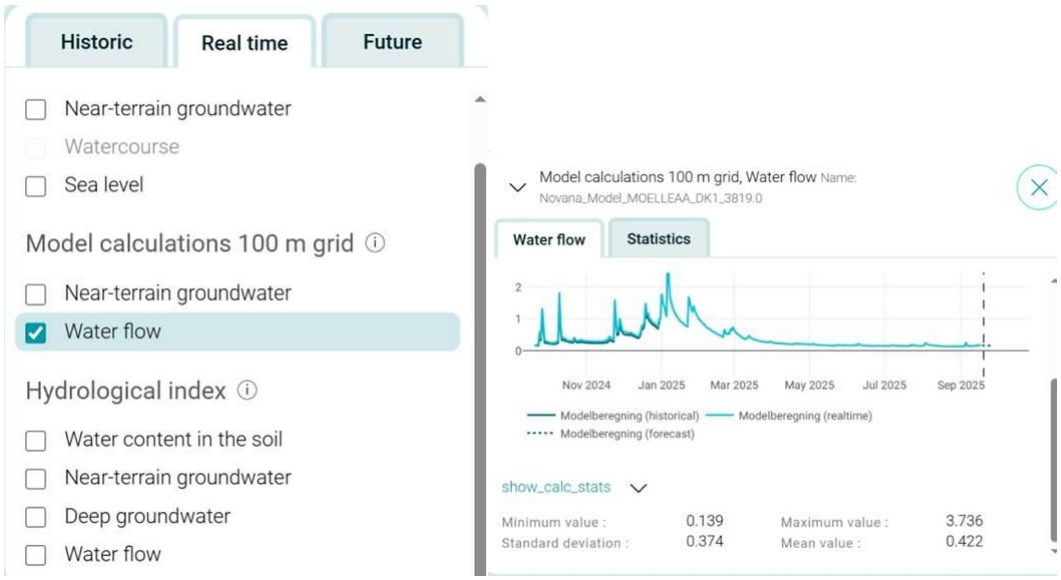
Integrate real-time monitoring data for CSO concentrations and flow. Develop dynamic models simulating time-varying discharges and pollutant decay. Perform sensitivity analyses on key parameters. Expand assessment to include multiple pollutants.

In conclusion, while the results reliably indicate CSO-related problems in Mølle Å, improvements in model complexity and data integration would strengthen trustworthiness. Lyngby Municipality should prioritize investments based on further refined assessments.

APPENDIX- STEP I-System and data description

APPENDIX-Task 1.1-System introduction

All flow data on Danish rivers are available on this website:
https://hip.dataforsyningen.dk/



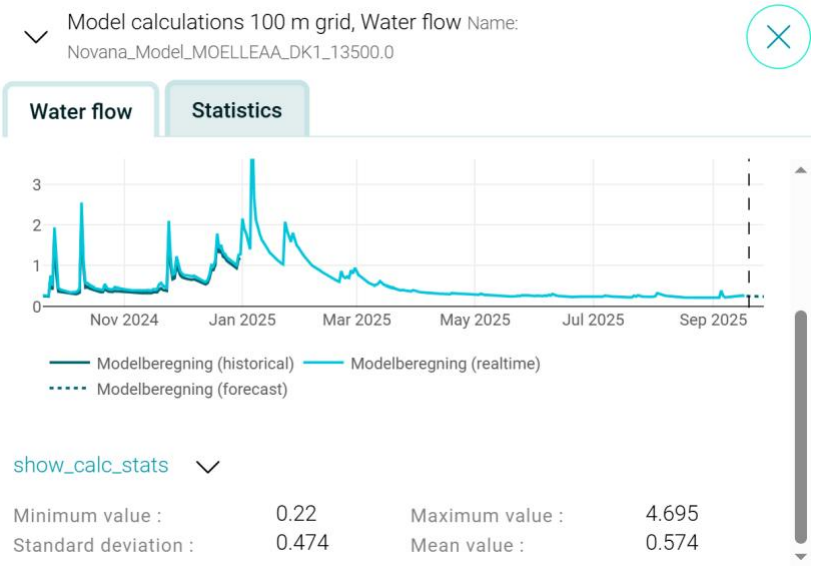
Water flow | Statistics

Statistics from 1990-2019 (m³/s)	total	winter	spring	summer	autumn	january	february
Middle	0.520	0.532	0.232	0.358	0.965	1.066	1.068
Minimum	0.099	0.140	0.099	0.106	0.153	0.208	0.198
Maximum	7.232	6.304	7.232	4.657	7.227	5.427	7.227

Statistics from 1990-2019 (m³/s)	march	april	may	june	july	august	september	october	november	december
Middle	0.866	0.464	0.264	0.227	0.245	0.223	0.269	0.304	0.502	0.770
Minimum	0.219	0.161	0.140	0.111	0.099	0.099	0.106	0.106	0.134	0.153
Maximum	6.304	4.642	2.606	5.688	7.232	4.398	4.657	2.863	3.176	5.070

River flow data can be found on the website. Data points were selected from the river's course starting at Lake Lingbi and extending to its estuary, enabling the calculation and assessment of the river's average flow rate and the months with the highest and lowest flow volumes.

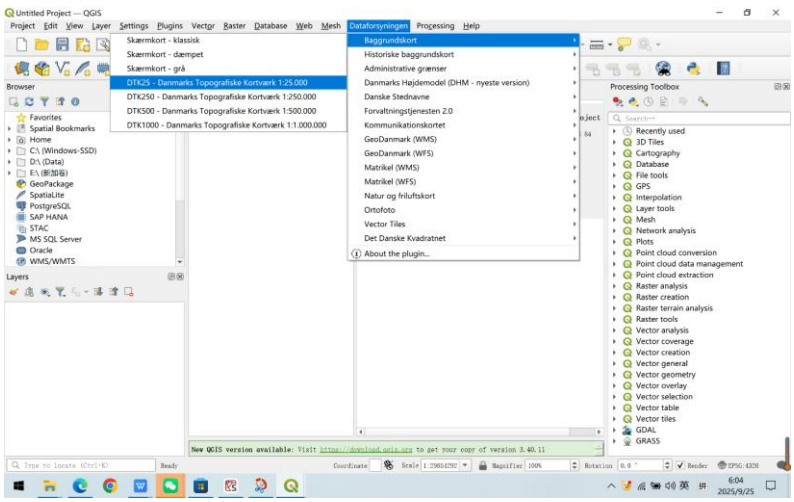
The image above displays the flow data from the data node where the lake enters the river. The image below displays the flow data from the data node where the river flows into the sea.



Statistics from 1990-2019 (m³/s)	total	winter	spring	summer	autumn	january	february
Middle	0.707	0.724	0.339	0.496	1.277	1.407	1.412
Minimum	0.163	0.216	0.163	0.167	0.226	0.288	0.277
Maximum	9.641	7.900	9.641	7.139	9.387	6.906	9.387

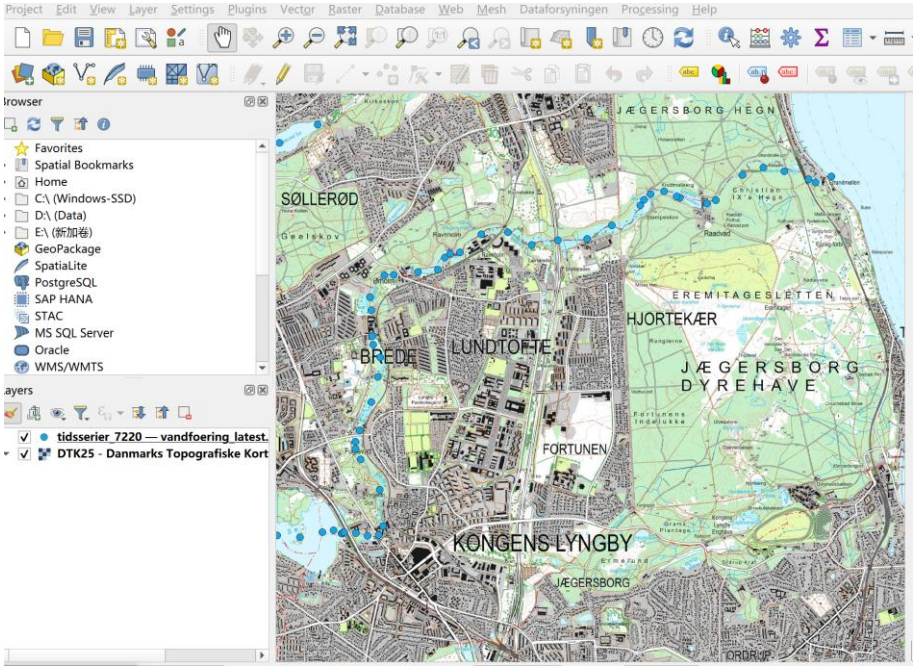
Statistics from 1990-2019 (m³/s)	march	april	may	june	july	august	september	october	november	december
Middle	1.153	0.639	0.378	0.332	0.356	0.330	0.394	0.423	0.674	1.023
Minimum	0.315	0.239	0.216	0.182	0.164	0.163	0.167	0.167	0.206	0.226
Maximum	7.900	6.391	3.870	8.334	9.641	7.096	7.139	4.161	4.385	6.649

Open QGIS and then select a map in QGIS as the base layer.



Add the CSO data on the map. Select: Layer->Add layer->Add vector layer. Select the .shp file with the

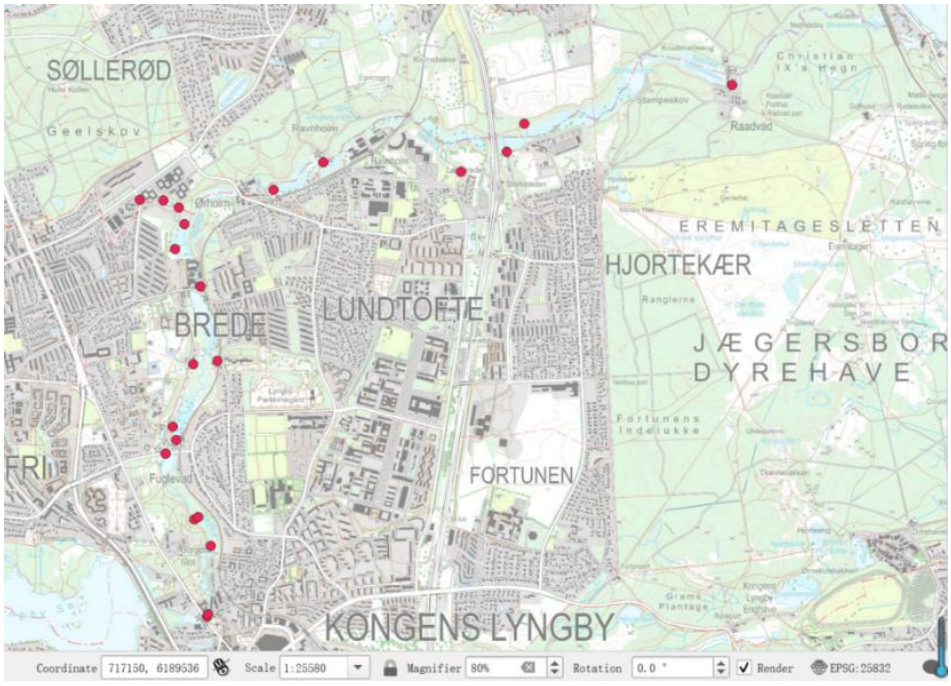
downloaded data.



Subsequently, input the equation for filtering.

The query expression to type should be
("Closed" = 'Aktivt' AND "Type" = 'Overløbsbygværk med bassin') OR ("Closed" = 'Aktivt' AND "Type" = 'Overløbsbygværk uden bassin').

Then 23 CSO structures were screened out.



Subsequently, Python was employed for data processing, and editing the code yielded the following results.

```
# Which CSO structures discharged the most?
max_row = csoData.loc[csoData['Vol_m3_yr'].idxmax()]
max_row
```

[145] ✓ 0.0s Python

```
OutfallId                bda213b5-6caa-4229-8ab2-e57b84fad124
Name                                                 Ly R16
ReducedAre                                             121.0
RetentionB                                            2200.0
Type                                Overlï¿½bsbygvï¿½rk med bassin
SewerStruc                                               OS
Authority                                LYNGBY-TAARBï¿½K KOMMUNE
Owner                LYNGBY-TAARBï¿½K SPILDEVAND A/S (CVR 32292771)
Started                                      1900-01-01T00:00:00Z
Closed                                                   Aktivt
LatestDisc                                               2024
Vol_m3_yr                                              245162
LatestDi_1                                              44129
LatestDi_2                                               7355
totN_kg_yr                                               2942
totP_kg_yr                                                490
DocumentUr    https://b0301proddatasto.blob.core.windows.net...
LatestDi_3                                              15.0
Name: 21, dtype: object
```

```
# how many structures
num_struc = len(csoData['Name'])
num_struc
```

[137] ✓ 0.0s Python

```
23
```

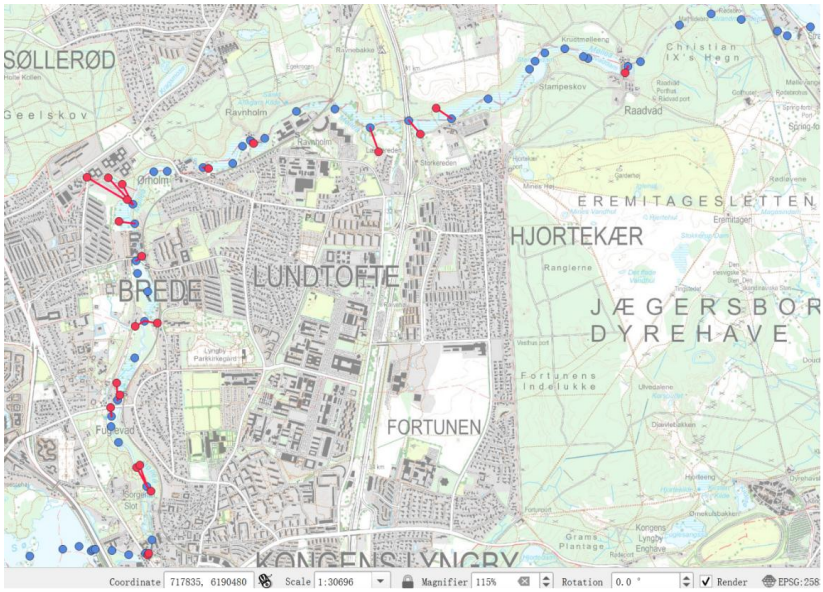
```
# how big were the discharged water volumes and the N mass
water_vol = csoData['Vol_m3_yr'].sum()
N_mass = csoData['totN_kg_yr'].sum()
print(water_vol)  # m3
print(N_mass)  # kg
```

[142] ✓ 0.0s Python

```
636156
7589
```

Select the following tools in the toolbar, choose the data source, and couple each CSO structure to the corresponding node in the river model.





APPENDIX-Task 1.2-Emissions of cumulative nitrogen pollutants

1. Preparatory work for data processing

```
#clean the workspace
from IPython import get_ipython
get_ipython().run_line_magic('reset', '-f')

#close all figures
import matplotlib.pyplot as plt
plt.close('all')
plt.rcParams['font.size']=8

#import the needed modules
import pandas as pd
import numpy as np

#read yearly, monthly water volume data and CSO data of the whole catchment from csv file
yflowData=pd.read_csv('the yearly data.csv', sep=",", decimal=".")
mflowData=pd.read_csv('the monthly data.csv', sep=",", decimal=".")
CSOData=pd.read_csv('finalCSOdata.csv', sep=",")
```

2. Calculate the yearly water volume entering the Mølle Å stream from the Lyngby Lake.

```
#extract the node 3687 to represent the outlet from the Lyngby Lake
ext3687=yflowData['beregningspunktlokalid'].str.contains('Novana_Model_MOELLEAA_DK1_3687')

yflowData_LyngbyLake=yflowData[ext3687]
yflowData_LyngbyLake=yflowData_LyngbyLake.reset_index(drop=True)

del(ext3687)

#task 1.2.1: calculate the yearly water volumn entering the Moelle aa stream from the Lyngby Lake
no_volume_data=len(yflowData_LyngbyLake)
yVolume_LyngbyLake=pd.DataFrame(np.zeros([no_volume_data,2],dtype=float),columns=['volume_m3','year'])

for i in range(no_volume_data):
    yflowValue=yflowData_LyngbyLake['vandfoering'][i]
    sc_to_yr=31536000  #second/year =365*24*60*60
    yVolume_LyngbyLake['volume_m3'][i]=yflowValue * sc_to_yr  #m3/s * s/y = m3/y
    yVolume_LyngbyLake['year'][i]=int(yflowData_LyngbyLake['aar'][i])

del(no_volume_data,i,yflowValue,sc_to_yr)
```

3. Make a water balance for yearly volumes of the Mølle Å stream, and find out its major source of the water volume discharged to the Øresund by ploting Figure 2.

```
#extract the node 13500 to represent the discharge to the Øresund
ext13500=yflowData['beregningspunktlokalid'].str.contains('Novana_Model_MOELLEAA_DK1_13500')

yflowData_toOeresund=yflowData[ext13500]
yflowData_toOeresund=yflowData_toOeresund.reset_index(drop=True)

del(ext13500)

#calculate the yearly water volume discharging to the Øresund
no_volume_data=len(yflowData_toOeresund)
yVolume_toOeresund=pd.DataFrame(np.zeros([no_volume_data,2],dtype=float),columns=['volume_m3','year'])

for i in range(no_volume_data):
    yflowValue=yflowData_toOeresund['vandfoering'][i]
    sc_to_yr=31536000  #second/year =365*24*60*60
    yVolume_toOeresund['volume_m3'][i]=yflowValue * sc_to_yr  #m3/s * s/y = m3/y
    yVolume_toOeresund['year'][i]=int(yflowData_toOeresund['aar'][i])

del(no_volume_data,i,yflowValue,sc_to_yr)

#calculate the CSO total volume in 2024 to represent its yearly volume
CSOVolume=CSOData['Vol_m3_yr'].sum()

#calculate the mean volumes of inflow, runoff and discharge, and the other yearly inputs
inflowVolume=yVolume_LyngbyLake['volume_m3'].mean()
dischargeVolume=yVolume_toOeresund['volume_m3'].mean()
runoffVolume=dischargeVolume-inflowVolume

print("The mean of yearly water volume entering the Moelle Aa stream from the Lyngby Lake is:",inflowVolume)

#task 1.2.2: plot the pie chart of discharge composition
parts=[inflowVolume,runoffVolume,CSOVolume]
labels=['Inflow','Natural runoff','CSO']

plt.figure(figsize=(3,3))
plt.pie(parts,labels=labels,autopct=lambda p:'{:.1f}%'.format(p),startangle=90)

plt.legend()
plt.title("Composition of Yearly Discharge")
plt.show()

del(parts,labels)
```

4. Make a mass balance for yearly nitrogen loads, and find out the major nitrogen source discharged by the Mølle Å stream to the Øresund by ploting Figure 3.

```
#task 1.2.3: make a mass balance for yearly nitrogen loads
concN_runoff=1.5  #mgN/l, given by Assignment 1 Instructions
totN_runoff=concN_runoff*runoffVolume*1000*10**(-6)  #mgN/l * m3 * 1000l/m3 * 10^(-6)kg/mg = kg

concN_LyngbyLake=0.11  #mgN/l, calculate the data average from Water Pollution course
totN_LyngbyLake=concN_LyngbyLake*inflowVolume*1000*10**(-6)

totN_CSO=CSOData['totN_kg_yr'].sum()  #kg

parts=[totN_LyngbyLake,totN_runoff,totN_CSO]
labels=['Inflow','Natural runoff','CSO']

plt.figure(figsize=(3,3))
plt.pie(parts,labels=labels,autopct=lambda p:'{:.1f}%'.format(p),startangle=90)

plt.legend()
plt.title("Composition of Yearly Nitrogen Load in Combined System")
plt.show()

del(parts,labels)
```

5. Make a mass balance for nitrogen in the case that all the CSO catchments are converted to separate sewer systems, and find out the major nitrogen source and reduced nitrogen loads in this scenario. Plot the Figure 4 to display the accounts of different nitrogen sources.

```
#task 1.2.4: make a mass balance for nitrogen in the seperate systems
totyRainfall=758.4  #mm, total yearly rainfall in 2024
CSOData['sep_vol_m3/yr']=CSOData['ReducedAre'] * 10**4 * totyRainfall * 10**(-3)  #ha * 10^4m2/ha * mm * 10^(-3)m/mm = m3

concN_sep=2  #mg/l
CSOData['sep_N_kg/yr']=CSOData['sep_vol_m3/yr'] * concN_sep * 10**(-3)  #m3 * mg/l * 10^(-6) kg/mg * 10^3 l/m3 = kg
    #all the drainage outlets along the river in separate systems are considered as stormwater outlets, no wastewater inflow
totN_stormwater=CSOData['sep_N_kg/yr'].sum()  #kg

parts=[totN_LyngbyLake,totN_runoff,totN_stormwater]
labels=['Inflow','Natural runoff','Stormwater']
plt.figure(figsize=(3,3))
plt.pie(parts,labels=labels,autopct=lambda p:'{:.1f}%'.format(p),startangle=90)
plt.legend()
plt.title("Composition of Yearly Nitrogen Load in Separate System")
plt.show()

totN_reduced=totN_CSO-totN_stormwater
print("The total N load reduce",totN_reduced,"kg/year in separate sewage system compared to combined sewage system.")

del(totyRainfall,concN_sep,parts,labels,totN_reduced)
```

APPENDIX-Task 1.3-Emission of micropollutants

Decide which micropollutants we will consider in the rest of the assignment. The results come out as the fifth one in the list- Benz(ghi)perylene.

```
studentNumbers=[252709,252639,252690,254721]
lastDigits=[num%100 for num in studentNumbers]
substanceToChoose=sum(lastDigits)/33
substanceToChoose=round(substanceToChoose)

print("The substance number of our group is:",substanceToChoose)
```

For measurement concentrations of nitrogen in Lyngby Lake, the results of mean, standard deviation and CV are calculated as follows. And then we compare the variability of this data set by CV with the Benz(ghi)perylene IQR/median ratio.

Site	1	2	3	4	5
Year	Furesø	before dam	after waterfall	Lyngby sø	Bagsværd sø
2014	0.015	0.06	0.12	0.07	0.11
2013	0.068	0.164	nd	0.0155	0.0335
2012	0.04	0.08	nd	0.02	0.055
2011	0.05	0.05	0.05	0.15	0.06
2010	0.025	0.025	0.03	0.03	< 0.05
2009	0.03	0.055	0.08	0.04	0.09
2008	0.053	0.203	0.065	0.173	0.327
2007	0.025	0.22	0.12	0.25	0.08
2006	0.06	0.02	0.05	0.01	0.05
2004	0.4	0.22	0.3	0.3	0.65
2003	0.15	0.11	0.07	0.15	nd
2002	0.19	0.09	0.05	0.075	nd
			mean	0.107	
			std	0.093	
			CV	0.870	

APPENDIX-STEP II-Model building

APPENDIX-Task 2.1-Conceptualization description of the model

The Corresponding Code:

```
# clean the workspace
from IPython import get_ipython
get_ipython().magic('reset -sf')
# close all figures
import matplotlib.pyplot as plt
plt.close('all')
import pandas as pd
import numpy as np
csoData=pd.read_csv('csoData.csv',sep=",",decimal=",",encoding='latin-1')
RiverData=pd.read_csv('RiverData.csv',sep=",",decimal=",",encoding='latin-1')
idxMOELLEAA=RiverData['beregningspunktlokalid'].str.contains('MOELLEAA')
RiverData_MOELLEAA=RiverData[idxMOELLEAA]
RiverData_MOELLEAA=RiverData_MOELLEAA.reset_index(drop=True)
idxYY=RiverData_MOELLEAA['aar'].astype(str).str.contains('2019')
RiverData_MOELLEAA_YY=RiverData_MOELLEAA[idxYY]
RiverData_MOELLEAA_YY=RiverData_MOELLEAA_YY.reset_index(drop=True)
idxMM=RiverData_MOELLEAA_YY['maaned'].str.contains('januar')
idxMM = RiverData_MOELLEAA_YY['maaned'] == 'januar'
RiverData_MOELLEAA_MM = RiverData_MOELLEAA_YY[idxMM]
RiverData_MOELLEAA_MM=RiverData_MOELLEAA_MM.reset_index(drop=True)
idxUp = RiverData_MOELLEAA_MM['beregningspunktlokalid'].astype(str).str.contains('3687')
RiverDataMM_Up = RiverData_MOELLEAA_MM[idxUp]
RiverDataMM_Up = RiverDataMM_Up.reset_index(drop=True)

idxDown = RiverData_MOELLEAA_MM['beregningspunktlokalid'].astype(str).str.contains('13500')
RiverDataMM_Down = RiverData_MOELLEAA_MM[idxDown]
RiverDataMM_Down = RiverDataMM_Down.reset_index(drop=True)
idxUp = 0
idxDown = len(RiverData_MOELLEAA_MM) - 1
N = idxDown - idxUp + 1
RiverQ = pd.DataFrame({
    "flow": RiverData_MOELLEAA_MM.loc[idxUp:idxDown, "vandfoering"].values,
    "nodeID": RiverData_MOELLEAA_MM.loc[idxUp:idxDown, "beregningspunktlokalid"].values,
    "X": RiverData_MOELLEAA_MM.loc[idxUp:idxDown, "X"].values,
    "Y": RiverData_MOELLEAA_MM.loc[idxUp:idxDown, "Y"].values,
    "distance": [None] * N,
    "Qadded": [None] * N
```

```
})
RiverC = pd.DataFrame({
    "concentration": [None] * N,     # used to store simulated concentration
    "X": RiverQ["X"],                 # copy X coordinates from RiverQ
    "Y": RiverQ["Y"],                 # copy Y coordinates from RiverQ
    "distance": RiverQ["distance"] # copy distance info from RiverQ
})
# Create EQS_exc (same structure as RiverC, N rows × 4 columns)
# Used for plotting exceedance values later
EQS_exc = pd.DataFrame({
    "columnWithEQSexcValues": [None] * N,     # can store values to be plotted
    "X": RiverQ["X"],                          # copy X coordinates from RiverQ
    "Y": RiverQ["Y"],                          # copy Y coordinates from RiverQ
    "distance": RiverQ["distance"]             # copy distance info from RiverQ
})
import re
for i in range(N):
    node_id = RiverQ['nodeID'][i]
    # check if contains 'downstream' (case-insensitive)
    if re.search(r'downstream', node_id, re.IGNORECASE):
        # extract number before 'downstream' (supports int/float, ignores spaces)
        match = re.search(r'(\d+\.?\d*)\s*downstream', node_id, re.IGNORECASE)
        if match:
            # extract number part and compute distance
            number_part = float(match.group(1))
            RiverQ.loc[i, 'distance'] = number_part - 3687
        else:
            # mark as NaN if no valid number found
            RiverQ.loc[i, 'distance'] = float('nan')
            print(f"Node {i}, nodeID '{node_id}' cannot extract number before downstream")
    else:
        # handle normal nodeID format (split by underscore)
        string_name = node_id.split('_')
        try:
            # take last part as number and compute distance
            RiverQ.loc[i, 'distance'] = float(string_name[-1]) - 3687
        except (ValueError, IndexError):
            # mark as NaN if conversion fails or no element
            RiverQ.loc[i, 'distance'] = float('nan')
            print(f"Node {i}, nodeID '{node_id}' format invalid")
# Key parameter settings (based on latest data)
C0 = 0.0   # initial concentration (μg/L) - updated with known value
```

```
RiverC.loc[0, 'conc'] = C0   # initial concentration for the first node
# Environmental Quality Standard and CSO parameters
EQS = 8.2e-3          # Environmental Quality Standard (µg/L)
t_CSO = 4.3           # overflow duration (hours)
theta = 0.0231        # annual volume fraction (2.31%)
CSO_conc_benzo = 0.020   # CSO benzo(ghi)perylene concentration (µg/L) - typical reference value
import re
def normalize_id(s):
    # remove _Downstream suffix if present
    base = s.split('_Downstream')[0]
    return base
RiverQ['nodeID_clean'] = RiverQ['nodeID'].apply(normalize_id)
def normalize_number(s):
    match = re.search(r"(-?\d+\.?\d*)$", s)
    if match:
        num = float(match.group(1))
        # round to integer and append ".0"
        num = int(round(num))
        return re.sub(r"(-?\d+\.?\d*)$", f"{num}.0", s)
    return s
RiverQ['nodeID_clean'] = RiverQ['nodeID_clean'].apply(normalize_number)
# intersection of two columns
common_ids = set(RiverQ['nodeID_clean']) & set(csoData['HubName'])
# filter matching nodes from RiverQ
# boolean column
isThereCSO = RiverQ['nodeID_clean'].isin(common_ids)
matched_nodes = RiverQ[isThereCSO]
csoData['Vol_m3_yr'] = pd.to_numeric(csoData['Vol_m3_yr'], errors='coerce')
RiverQ['flow'] = pd.to_numeric(RiverQ['flow'], errors='coerce')
# 1. Initialization
RiverQ['flow'] = RiverQ['flow'] * 3600
RiverQ['Qadded'] = 0.0
RiverC['conc'] = 0.0   # or pd.NA
N = len(RiverQ)
# 2. Loop
for i in range(N):
    # find CSO corresponding to the current node
    isThereCSO = csoData['HubName'].isin([RiverQ.loc[i, 'nodeID']])
    idxCSO = np.where(isThereCSO)[0]
    CSO_flux = 0
    CSO_Qtot_current = 0
    # accumulate CSO at current node
```

```
    for j in idxCSO:
        V_CSO = csoData.loc[j, 'Vol_m3_yr'] * theta
        Q_CSO = V_CSO / t_CSO
        CSO_flux += Q_CSO * CSO_conc_benzo
        CSO_Qtot_current += Q_CSO
    # accumulate Qadded
    if i == 0:
        RiverQ.loc[i, 'Qadded'] = CSO_Qtot_current
    else:
        RiverQ.loc[i, 'Qadded'] = RiverQ.loc[i - 1, 'Qadded'] + CSO_Qtot_current
    # update concentration
    if i == 0:
        prev_total_flow = RiverQ.loc[i, 'flow']
        prev_conc = C0
    else:
        prev_total_flow = RiverQ.loc[i - 1, 'flow'] + RiverQ.loc[i - 1, 'Qadded']
        prev_conc = RiverC.loc[i - 1, 'conc']
    current_total_flow = RiverQ.loc[i, 'flow'] + RiverQ.loc[i, 'Qadded']
    RiverC.loc[i, 'conc'] = (prev_conc * prev_total_flow + CSO_flux) / current_total_flow
EQS_exc["columnWithEQSexcValues"] = RiverC["conc"].where(RiverC["conc"] > EQS, other=pd.NA)
plt.figure()
plt.rcParams.update({'font.size': 22})
RiverC['distance'] = RiverQ['distance']
plt.plot(RiverC['distance'], RiverC['conc'], 'o', color='black')
plt.xlabel("Distance [m]")
plt.ylabel("Concentration [µg/L]")
plt.grid()
plt.figure()
plt.rcParams.update({'font.size': 22})
plt.plot(RiverC['distance'], EQS_exc["columnWithEQSexcValues"], 'o', color='black')
plt.xlabel("Distance [m]")
plt.ylabel("EQS Exceedance")
plt.grid()
plt.show()
EQS_exc.to_csv("EQS_exc.csv", index=False)
RiverQ.to_csv("RiverQ.csv", index=False)
RiverC.to_csv("RiverC.csv", index=False)
```

APPENDIX-Task 2.2-Model diagnosis and testing

Identify the key node:

```
# Target distance value (replaces the original node name)
target_distance = 396    # The distance value to search for
```

```
# Filter indices based on the 'distance' column (ensure numerical matching, not string matching)
key_idx = RiverC[RiverC['distance'] == target_distance].index
# Check if the corresponding index is found
if len(key_idx) > 0:
    key_idx = key_idx[0]  # Take the first matching index (if there are multiple matches)
    # Retrieve the concentration and distance corresponding to this distance from RiverC
    # (The distance itself is the target value; target_distance can be used directly here)
    key_distance = RiverC.loc[key_idx, 'distance']  # Alternatively, use target_distance directly
    key_conc = RiverC.loc[key_idx, 'conc']
    # Retrieve the EQS exceedance value corresponding to the index from EQS_exc
    key_eqs = EQS_exc.loc[key_idx, "columnWithEQSexcValues"]
else:
    # Throw an error if the distance value is not found
    raise ValueError(f"Distance value {target_distance} not found in RiverC")
# Figure 1: Concentration distribution (mark the point where distance = 396)
plt.figure()
plt.rcParams.update({'font.size': 22})
plt.plot(RiverC['distance'], RiverC['conc'], 'o', color='black', label='Concentration')
plt.plot(key_distance, key_conc, 'o', color='red', markersize=10, label=f'Distance = {target_distance}')
# Label the distance value next to the point (replaces the original node name)
plt.text(key_distance, key_conc + 0.001, f'distance: {target_distance}', rotation=45, fontsize=18)
plt.xlabel("Distance")
plt.ylabel("Concentration [µg/L]")
plt.grid()
plt.legend()  # Display legend
plt.show()
# Figure 2: EQS exceedance (mark the point where distance = 396)
plt.figure()
plt.rcParams.update({'font.size': 22})
plt.plot(RiverC['distance'], EQS_exc["columnWithEQSexcValues"], 'o', color='black', label='EQS
Exceedance')
plt.plot(key_distance, key_eqs, 'o', color='red', markersize=10, label=f'Distance = {target_distance}')
# Label the distance value next to the point
plt.text(key_distance, key_eqs + 0.001, f'distance: {target_distance}', rotation=45, fontsize=18)
plt.xlabel("Distance")
plt.ylabel("EQS Exceedance")
plt.grid()
plt.legend()  # Display legend
plt.show()
```

APPENDIX-Task 2.3-Local sensitivity analysis

```python

import pandas as pd
import numpy as np
import re
from scipy.stats import lognorm,norm
import matplotlib.pyplot as plt
### Task2.3
def run_model(csoData,RiverData,param):

    # check number of parameters
    if len(param)!=5:
        errorSrt=('ERROR! \nthe number of model parameters is wrong' +
                  '\n3 were expected - you provided ' + str(len(param) ))
        print(errorSrt)
        return
    else:
        C0 = param[0]   # [ug/l]
        EQS = param[1] # [ug/l]
        t_CSO = param[2] # [h]
        theta = param[3] #[%]
        CSO_conc_benzo = param[4] # [ug/l]

    # select data
    idxMOELLEAA = RiverData['beregningspunktlokalid'].str.contains('MOELLEAA')
    RiverData_MOELLEAA = RiverData[idxMOELLEAA].reset_index(drop=True)

    idxYY = RiverData_MOELLEAA['aar'].astype(str).str.contains('2019')
    RiverData_MOELLEAA_YY = RiverData_MOELLEAA[idxYY].reset_index(drop=True)

    idxMM = RiverData_MOELLEAA_YY['maaned'] == 'januar'
    RiverData_MOELLEAA_MM = RiverData_MOELLEAA_YY[idxMM].reset_index(drop=True)
```

```
33
34      idxUp = 0
35      idxDown = len(RiverData_MOELLEAA_MM) - 1
36      N = idxDown - idxUp + 1
37
38      # DataFrame
39      RiverQ = pd.DataFrame({
40          "flow": RiverData_MOELLEAA_MM.loc[idxUp:idxDown, "vandfoering"].values,
41          "nodeID": RiverData_MOELLEAA_MM.loc[idxUp:idxDown, "beregningspunktlokalid"].values,
42          "X": RiverData_MOELLEAA_MM.loc[idxUp:idxDown, "X"].values,
43          "Y": RiverData_MOELLEAA_MM.loc[idxUp:idxDown, "Y"].values,
44          "distance": [None]*N,
45          "Qadded": [0.0]*N
46      })
47
48      RiverC = pd.DataFrame({
49          "conc": np.zeros(N),
50          "X": RiverQ["X"],
51          "Y": RiverQ["Y"],
52          "distance": RiverQ["distance"]
53      })
54
55      EQS_exc = pd.DataFrame({
56          "columnWithEQSexcValues": [None]*N,
57          "X": RiverQ["X"],
58          "Y": RiverQ["Y"],
59          "distance": RiverQ["distance"]
60      })
61
```

```
62      # calculate distances
63      for i in range(N):
64          node_id = RiverQ['nodeID'][i]
65          if re.search(r'downstream', node_id, re.IGNORECASE):
66              match = re.search(r'(\d+\.?\d*)\s*downstream', node_id, re.IGNORECASE)
67              if match:
68                  number_part = float(match.group(1))
69                  RiverQ.loc[i, 'distance'] = number_part - 3687
70              else:
71                  RiverQ.loc[i, 'distance'] = float('nan')
72          else:
73              string_name = node_id.split('_')
74              try:
75                  RiverQ.loc[i, 'distance'] = float(string_name[-1]) - 3687
76              except:
77                  RiverQ.loc[i, 'distance'] = float('nan')
78
```

```
    # clean nodeID, match CSO
    for i in range(N):
        node_id = RiverQ['nodeID'][i]
        if re.search(r'downstream', node_id, re.IGNORECASE):
            match = re.search(r'(\d+\.?\d*)\s*downstream', node_id, re.IGNORECASE)
            if match:
                number_part = float(match.group(1))
                RiverQ.loc[i, 'distance'] = number_part - 3687
            else:
                RiverQ.loc[i, 'distance'] = float('nan')
        else:
            string_name = node_id.split('_')
            try:
                RiverQ.loc[i, 'distance'] = float(string_name[-1]) - 3687
            except:
                RiverQ.loc[i, 'distance'] = float('nan')

    csoData['Vol_m3_yr'] = pd.to_numeric(csoData['Vol_m3_yr'], errors='coerce')
    RiverQ['flow'] = pd.to_numeric(RiverQ['flow'], errors='coerce') * 3600  # m3/h

```

```
    # loop calculation
    RiverC.loc[0, 'conc'] = C0
    for i in range(N):
        isThereCSO = csoData['HubName'].isin([RiverQ.loc[i, 'nodeID']])
        idxCSO = np.where(isThereCSO)[0]

        CSO_flux = 0
        CSO_Qtot_current = 0
        for j in idxCSO:
            V_CSO = csoData.loc[j, 'Vol_m3_yr'] * theta
            Q_CSO = V_CSO / t_CSO
            CSO_flux += Q_CSO * CSO_conc_benzo
            CSO_Qtot_current += Q_CSO

        RiverQ.loc[i, 'Qadded'] = RiverQ.loc[i-1, 'Qadded'] + CSO_Qtot_current if i > 0 else CSO_Qtot_current

        prev_total_flow = RiverQ.loc[i-1, 'flow'] + RiverQ.loc[i-1, 'Qadded'] if i > 0 else RiverQ.loc[i, 'flow']
        prev_conc = RiverC.loc[i-1, 'conc'] if i > 0 else C0

        current_total_flow = RiverQ.loc[i, 'flow'] + RiverQ.loc[i, 'Qadded']
        RiverC.loc[i, 'conc'] = (prev_conc * prev_total_flow + CSO_flux) / current_total_flow

```

```
121
122     # calculate EQS
123     EQS_exc["columnWithEQSexcValues"] = RiverC["conc"].where(RiverC["conc"] > EQS, other=pd.NA)
124     RiverC["distance"] = RiverQ["distance"]
125
126     # calculate pollutant loads(g/d)
127     # RiverQ['flow'] 单位 m3/h, RiverC['conc'] 单位 µg/L
128     # Load [g/d] = conc[µg/L] * flow[m3/h] * 0.024
129     RiverC['load_g_per_d'] = RiverC['conc'] * RiverQ['flow'] * 0.024
130
```

```
131     return RiverC, EQS_exc
132 # load measured values from csv file
133 RiverData = pd.read_csv('/Users/ganyining/Desktop/DTU_ Master/Environment_modelling/Model_1/Assignment_1/data/the monthly data.csv',
134                         sep=",",decimal=",",encoding='latin-1')
135 csoData = pd.read_csv('/Users/ganyining/Desktop/DTU_ Master/Environment_modelling/Model_1/Assignment_1/data/finalCSOdata2.0.csv',
136                       sep=",",decimal=",",encoding='latin-1')
137 # initial parameter value
138 param = np.zeros(5)
139 param[0] = 0.0  # Co
140 param[1] = 8.2e-3
141 param[2] = 4.3
142 param[3] = 0.0231
143 param[4] = 0.020
144 pd.DataFrame(param)
```

```
145 # calculate initial dataframe(RiverC, EQS_exc)
146 out = run_model(csoData,RiverData,param)
147 print(out[0]) # RiverC
148 print(out[1]) # EQS_exc
149 # output 1: The number of river nodes exceeding the EQS
150 num_EQS_exc = out[1]['columnWithEQSexcValues'].count()
151 # output 2: The max concentration
152 max_conc = out[0]['conc'].max()
153 # output3: pollutant loads discharged to the sea
154 MP_load = out[0]['load_g_per_d'].iloc[-1]
155 parSample = np.tile(param,(6,1))
156 parSample_matrix = pd.DataFrame(parSample)
157 parSample_matrix
158 def make_parSample(param, dPar):
159     param = np.array(param, dtype=float)
160     n = len(param) + 1
161     parSample = np.zeros((n, len(param)))
162     parSample[0, :] = param
163     for i in range(len(param)):
164         parSample[i+1, :] = param.copy()
165         parSample[i+1, i] = max(param[i], 1e-3) * (1 + dPar)
166
167     return parSample
168
```

```
170    pd.DataFrame(make_parSample(param, 0.1))
171    # change parameter
172    par = make_parSample(param,0.1)
173    out0 = run_model(csoData,riverData,par[1,:])
174    out1 = run_model(csoData,riverData,par[2,:])
175    out2 = run_model(csoData,riverData,par[3,:])
176    out3 = run_model(csoData,riverData,par[4,:])
177    out4 = run_model(csoData,riverData,par[5,:])
178     (variable) num_EQS_exc0: int | Any
179    num_EQS_exc0 = out0[1]['columnWithEQSexcValues'].count()
180    max_conc0 = out0[0]['conc'].max()
181    MP_load0 = out0[0]['load_g_per_d'].iloc[-1]
182
183    # change EQS
184    num_EQS_exc1 = out1[1]['columnWithEQSexcValues'].count()
185    max_conc1 = out1[0]['conc'].max()
186    MP_load1 = out1[0]['load_g_per_d'].iloc[-1]
187
188    # change t_CSO
189    num_EQS_exc2 = out2[1]['columnWithEQSexcValues'].count()
190    max_conc2 = out2[0]['conc'].max()
191    MP_load2 = out2[0]['load_g_per_d'].iloc[-1]
192
193    # change theta
194    num_EQS_exc3 = out3[1]['columnWithEQSexcValues'].count()
195    max_conc3 = out3[0]['conc'].max()
196    MP_load3 = out3[0]['load_g_per_d'].iloc[-1]
197
198    # change CSO_conc_benzn
199    num_EQS_exc4 = out4[1]['columnWithEQSexcValues'].count()
200    max_conc4 = out4[0]['conc'].max()
201    MP_load4 = out4[0]['load_g_per_d'].iloc[-1]
```

```
202  # Sensitivity indices
203  # output 1: The number of river nodes exceeding the EQS
204  Si_num_EQS_exc = np.zeros(5)
205  Si_num_EQS_exc[0] = ((num_EQS_exc-num_EQS_exc0)/num_EQS_exc)/0.1
206  Si_num_EQS_exc[1] = ((num_EQS_exc-num_EQS_exc1)/num_EQS_exc)/0.1
207  Si_num_EQS_exc[2] = ((num_EQS_exc-num_EQS_exc2)/num_EQS_exc)/0.1
208  Si_num_EQS_exc[3] = ((num_EQS_exc-num_EQS_exc3)/num_EQS_exc)/0.1
209  Si_num_EQS_exc[4] = ((num_EQS_exc-num_EQS_exc4)/num_EQS_exc)/0.1
210  print(abs(Si_num_EQS_exc))
211
212  # Sensitivity indices
213  # output 2: The max concentration
214  Si_max_conc = np.zeros(5)
215  Si_max_conc[0] = ((max_conc-max_conc0)/max_conc)/0.1
216  Si_max_conc[1] = ((max_conc-max_conc1)/max_conc)/0.1
217  Si_max_conc[2] = ((max_conc-max_conc2)/max_conc)/0.1
218  Si_max_conc[3] = ((max_conc-max_conc3)/max_conc)/0.1
219  Si_max_conc[4] = ((max_conc-max_conc4)/max_conc)/0.1
220  print(abs(Si_max_conc))
221
222  # Sensitivity indices
223  # output3: pollutant loads discharged to the sea
224  Si_MP_load = np.zeros(5)
225  Si_MP_load[0] = ((MP_load-MP_load0)/MP_load)/0.1
226  Si_MP_load[1] = ((MP_load-MP_load1)/MP_load)/0.1
227  Si_MP_load[2] = ((MP_load-MP_load2)/MP_load)/0.1
228  Si_MP_load[3] = ((MP_load-MP_load3)/MP_load)/0.1
229  Si_MP_load[4] = ((MP_load-MP_load4)/MP_load)/0.1
230  print(abs(Si_MP_load))
231
```

APPENDIX-STEP III-Model application

APPENDIX-Task 3.1-Simulation results

Inputting the model from task 2.2 with an MP concentration of 0.18 will yield the model.

```
 # 关键参数设置（基于最新数据）
C0 = 0.0  # 初始浓度 (μg/L) - 已更新为已知值
RiverC.loc[0, 'conc'] = C0  # 第一个节点的初始浓度

# 环境质量标准与CSO参数
EQS = 8.2e-3  # 环境质量标准 (μg/L)
t_CSO = 4.3  # 溢流持续时间 (小时)
theta = 0.0231  # 年体积分数 (2.31%)
# %%
CSO_conc_benzo = 0.018  # CSO中苯并(ghi)苝浓度 (μg/L) - 参考典型值
```

（ （ Task 2.2 Model）

```
 # 关键参数设置（基于最新数据）
C0 = 0.18 # 初始浓度 (μg/L) - 已更新为已知值
RiverC.loc[0, 'conc'] = C0  # 第一个节点的初始浓度

# 环境质量标准与CSO参数
EQS = 8.2e-3  # 环境质量标准 (μg/L)
t_CSO = 4.3  # 溢流持续时间 (小时)
theta = 0.0231  # 年体积分数 (2.31%)
CSO_conc_benzo = 0.18  # CSO中苯并(ghi)苝浓度 (μg/L) - 参考典型值
```

（New Model）

APPENDIX-Task 3.2-Results Robustness

```
    # 核心计算循环
    for i in range(N):
        # 找当前节点对应的CSO
        isThereCSO = csoData['HubName'].isin([RiverQ.loc[i, 'nodeID']])
        idxCSO = np.where(isThereCSO)[0]

        CSO_flux = 0
        CSO_Qtot_current = 0
        for j in idxCSO:
            V_CSO = csoData.loc[j, 'Vol_m3_yr'] * theta
            Q_CSO = V_CSO / t_CSO
            # 每次都从分布随机采样
            CSO_conc_random = ben_dis.rvs()  # 单个值
            CSO_flux += Q_CSO * CSO_conc_random
            CSO_Qtot_current += Q_CSO

        # 累加Qadded
        RiverQ.loc[i, 'Qadded'] = RiverQ.loc[i-1, 'Qadded'] + CSO_Qtot_current if i > 0 else CSO_Qtot_current

        prev_total_flow = RiverQ.loc[i-1, 'flow'] + RiverQ.loc[i-1, 'Qadded'] if i > 0 else RiverQ.loc[i, 'flow']
        prev_conc = RiverC.loc[i-1, 'conc'] if i > 0 else C0

        current_total_flow = RiverQ.loc[i, 'flow'] + RiverQ.loc[i, 'Qadded']
        RiverC.loc[i, 'conc'] = (prev_conc * prev_total_flow + CSO_flux) / current_total_flow

    return RiverC # output of the function will be the simulated concentration
```

```python
### Taks 3.2
# data from the Miljøstyrelsen report
# concentration of Benz(ghi)perylen
x1 = 0.005  # [ug/l]
p1 = 0.05   # 5% percentile
x2 = 0.100  # [ug/l]
p2 = 0.95   # 95% percentile

x1 = np.log(x1)
x2 = np.log(x2)
p1ppf = norm.ppf(p1)
p2ppf = norm.ppf(p2)

# estimate the distribution parameters
sigma = (x2-x1)/(p2ppf-p1ppf)  # standard deviation
mean = ((x1 * p2ppf)-(x2 * p1ppf))/(p2ppf-p1ppf)  # mean value

ben_dis = lognorm(s=sigma, scale=np.exp(mean))
Sample100 = ben_dis.rvs(size=100)
Sample1000 = ben_dis.rvs(size=1000)
Sample10000 = ben_dis.rvs(size=10000)
print("%0.4f,%0.4f,%0.4f"%(np.percentile(Sample100,5), np.percentile(Sample100,50), np.percentile(Sample100,95)))
print("%0.4f,%0.4f,%0.4f"%(np.percentile(Sample1000,5), np.percentile(Sample1000,50), np.percentile(Sample1000,95)))
print("%0.4f,%0.4f,%0.4f"%(np.percentile(Sample10000,5), np.percentile(Sample10000,50), np.percentile(Sample10000,95)))
def riverModel(RiverQ, csoData, CSO_conc_benzo, C0=0.0, EQS=0.0082, t_CSO=4.3, theta=0.0231):

    N = len(RiverQ)
    RiverC = pd.DataFrame({
        "conc": np.zeros(N),
        "X": RiverQ["X"],
        "Y": RiverQ["Y"],
        "distance": RiverQ["distance"]
    })

    RiverC.loc[0, 'conc'] = C0
```

```python
#  RiverQ
idxMOELLEAA = RiverData['beregningspunktlokalid'].str.contains('MOELLEAA')
RiverData_MOELLEAA = RiverData[idxMOELLEAA].reset_index(drop=True)

(variable) RiverData_MOELLEAA_YY: DataFrame
str.contains('2019')
RiverData_MOELLEAA_YY = RiverData_MOELLEAA[idxYY].reset_index(drop=True)

idxMM = RiverData_MOELLEAA_YY['maaned'] == 'januar'
RiverData_MOELLEAA_MM = RiverData_MOELLEAA_YY[idxMM].reset_index(drop=True)

idxUp = 0
idxDown = len(RiverData_MOELLEAA_MM) - 1
N = idxDown - idxUp + 1

RiverQ = pd.DataFrame({
    "flow": pd.to_numeric(RiverData_MOELLEAA_MM.loc[idxUp:idxDown, "vandfoering"], errors='coerce') * 3600, # m3/h
    "nodeID": RiverData_MOELLEAA_MM.loc[idxUp:idxDown, "beregningspunktlokalid"].values,
    "X": RiverData_MOELLEAA_MM.loc[idxUp:idxDown, "X"].values,
    "Y": RiverData_MOELLEAA_MM.loc[idxUp:idxDown, "Y"].values,
    "distance": [None]*N,
    "Qadded": [0.0]*N
})
n_MC = 10  # number of Monte Carlo simulations
N = len(RiverQ)
MCsim = np.zeros([N, n_MC])
CSO_sample = ben_dis.rvs(size=n_MC)

for i in range(n_MC):
    CSO_conc_i = CSO_sample[i]
    out = riverModel(RiverQ, csoData, CSO_conc_i, C0=0.02, EQS=0.0082, t_CSO=4.3, theta=0.0231)
    MCsim[:,i]=out['conc']
```

```
327
328  # analysis results
329  q05 = np.percentile(MCsim, 5, axis=1)
330  q50 = np.percentile(MCsim, 50, axis=1)
331  q95 = np.percentile(MCsim, 95, axis=1)
332
333  # visualization
334  t = np.arange(N)
335  plt.figure(figsize=(10,6))
336  plt.plot(t, MCsim, color=(0.7,0.7,0.7), alpha=0.5)
337  plt.plot(t, q05, 'r-', label='5% percentile')
338  plt.plot(t, q50, 'b-', label='Median')
339  plt.plot(t, q95, 'r--', label='95% percentile')
340  plt.axhline(EQS, color='k', linestyle=':', linewidth=2, label='EQS standard')
341  plt.xlabel('River nodes')
342  plt.ylabel('Concentration [ug/L]')
343  plt.legend()
344  plt.show()
345  # visualization with EQS and always-exceed highlight
346  EQS = 0.0082
347  t = np.arange(N)
348  plt.figure(figsize=(10,6))
349
350  plt.plot(t, MCsim, color=(0.7,0.7,0.7), alpha=0.5)
351
352  plt.plot(t, q05, 'r-', label='5% percentile')
353  plt.plot(t, q50, 'b-', label='Median')
354  plt.plot(t, q95, 'r--', label='95% percentile')
355
356  plt.axhline(EQS, color='k', linestyle=':', linewidth=2, label='EQS')
357
358  always_exceed_idx = np.where(q05 > EQS)[0]
359  if len(always_exceed_idx) > 0:
360      plt.axvspan(always_exceed_idx[0], always_exceed_idx[-1], color='red', alpha=0.2, label='Always exceeding EQS')
361
362  plt.xlabel('River nodes')
363  plt.ylabel('Concentration [ug/L]')
364  plt.legend()
365  plt.show()
```

APPENDIX-Task 3.3-Results Reliability

Summarise the reliability of the results based on the findings from task 3.1 and 3.3.

Group Declaration

Task List	Anqi Yu	Qiaowen You	Jiayue Che	Yining Gan
Task 1.1	√			
Task 1.2		√		
Task 1.3		√		
Task 2.1		√(20%)	√(60%)	√(20%)
Task 2.2			√	
Task 2.3				√
Task 3.1	√(50%)		√(50%)	
Task 3.2				√
Task 3.3	√			
Writing	√	√	√	√
Graphs & Figures	√	√	√	√
Polishing & Quality Control	√	√	√	√

Declare for Use of AI

Task List	How was Generative AI used? (1, 2, 3, 4) *	Reflections on what was learned by the use of Generative AI
Task 1.1	1	/
Task 1.2	3	learned some new code to achieve the goals
Task 1.3	2	learned how to compare the variability of data sets with different presentation form
Task 2.1	3	learned some new code and its meaning
Task 2.2	3	corrected the grammar of my text
Task 2.3	3	Learned some new code to achieve the goals
Task 3.1	1	/
Task 3.2	3	Learned some new code to achieve the goals
Task 3.3	2	learned how to conduct evaluations more reasonably
Graphs & Figures	1	/
Polishing & Quality Control	1	/

* Uses of GenAI:

1 No use of GenAI

2 GenAI used for idea generation and structuring the assignment

3 GenAI to edit language/grammar but not content

4 GenAI used to write part of the text/make figures